Lab Assignment 2: Dynamic Branch Prediction

1 Objective

The objective of this assignment is to investigate the performance of different dynamic branch prediction schemes. All work on this assignment is to be done in groups of two.

This assignment involves a bonus competition aspect; you are encouraged to start early!

2 Problem Statement

Use the ChampSim [2] simulator to determine the number of mispredictions per thousand instructions (MPKI) of the following predictors:

- a 2-bit saturating counter predictor (see Figure 3.18 from the 2012 textbook [3] or Figure 2.18 from the 2025 textbook [4])
- a two-level predictor that uses 2-bit saturating counters (see PAp predictor in Figures 3.20 [3] or 2.20 [4])
- an open-ended predictor (with additional bonus marks for the top-performers)

Simulate all three predictors on the following benchmarks: `mcf`, `roms`, `charlie`, `merced`, `k-clique`, `sssp`, `llama2`, and `whisper`. Instructions on how to build the simulator and run these benchmarks are given in Section 4.

Create one short microbenchmark of 10-20 lines of C code to verify that the correct statistics are being collected for the two-level predictor. Though you will only be submitting one for the two-level predictor, you are still advised to write microbenchmarks to verify the correctness of the other predictors.

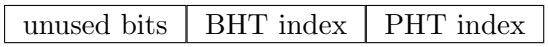
In addition, provide area, delay, and power measurements for your two-level and open-ended branch predictors using CACTI [1, 6, 8]. Section 3 provides a detailed walk-through example.

2.1 Dynamic Branch Predictors

The dynamic predictors (excluding the open-ended predictor) were discussed in class and are also described in Section 3.4.3 of your textbook.

For the 2-bit saturating counter predictor, assume that your prediction tables have 8192 bits.

For the two-level predictor, use the PAp predictor from Figure 3.20 [3]. The first-level is a 512-entries table of local history registers (BHT), with 6 history bits per entry. The second-level has sixteen pattern history tables (PHTs), with a 2-bit saturating counter per entry. The BHT and PHT table indices should be computed from the branch PC as follows:



Use `weak not-taken` as the initial state of your saturating counters.

2.2 Open-Ended Branch Predictor

The open-ended predictor design can be your own creation or inspired by previous research. You can use up to **128 Kbits** of storage for your open-ended predictor. Note: 128Kbits of storage refers

to the hardware storage that would be required to build your predictor in a processor, not the amount of memory your software simulator requires to create the open-ended predictor. Searching for "branch prediction" on `scholar.google.com` will provide you with quite a few references.

The open-ended predictor must have less than 3.35 geo-mean mispredictions per thousand instructions (MPKI) across the eight provided benchmarks, in order to receive marks on this portion of the assignment. Marks will be based on performance and implementation creativity. Simply making a larger version of one of the prescribed predictors will not merit any marks for creativity. In addition, your open-ended predictor implementation will compete against those of your classmates. Up to 1.5 bonus points will be awarded to the top ∼10 open-ended predictors.

3 Area, Delay and Power of different Branch Predictors

Branch prediction is critical to processor performance. A common rule of thumb is that one out of five instructions is a branch. A successful branch prediction scheme should (1) be accurate most of the time (i.e., have few mispredictions), (2) provide fast predictions (i.e., needing ten clock cycles to predict a branch is not acceptable), and (3) be within a reasonable hardware budget.

Computer architects always work within a given set of constraints. All design decisions have their own trade-offs. For example, it might be that requiring 1MB of storage for a branch predictor scheme yields a miniscule misprediction rate. However, it is highly unlikely anyone would dedicate that much chip real estate.

Even if storage were not the issue, larger structures are also slower to access. "Fitting" such a longer access time within a pipeline stage, could force you to lower your processor's frequency, thus making your entire system slower.

Finally, large structures, and especially fully-associative structures, are known to be very power-hungry. One of the challenges of moving towards smaller process sizes (e.g., AMD's Zen5 uses 4nm technology [7]) is that the static (i.e., leakage) power becomes considerable.

To better enhance your understanding of these trade-offs, we will use CACTI 6.5 to estimate the area, power, and delay of your branch prediction structures. This is especially critical for your open-ended branch predictor, as it will allow you to grasp how unrealistic some designs can be.

3.1 Using CACTI 6.5

CACTI is an integrated tool which models "cache access time, cycle time, area, leakage and dynamic power" [1, 6, 8]. Although CACTI was developed for analyzing caches, it can be used to approximate any storage structures you might have within your processor. You need two files to run CACTI: the CACTI executable and a configuration file. You can obtain these as follows:

```
cd ~/ece552
mkdir CACTI
cd CACTI

cp /cad2/ece552f/CACTI/cacti .
cp /cad2/ece552f/CACTI/cache.cfg .
cp /cad2/ece552f/CACTI/pureRAM.cfg .
```

We have provided you with two configuration files. The first file, `cache.cfg`, models a cache-like data structure, with a tag and a data array. The second file, `pureRAM.cfg`, models an untagged memory structure. Depending on your configuration you might prefer to use one over the other.

You can run CACTI by passing the configuration file as a parameter.

```
cacti -infile cache.cfg
```

This command prints out the area, delay and power info for a predetermined cache configuration. Feel free to redirect the output to a file for you convenience. Each configuration file provides a variety of options. The ones you are allowed to change are annotated with a **#ECE552: Modify if needed** comment.

The following two options are useful for debugging purposes:

- "Print input parameters": prints all your configuration settings when set to true.
- "Print level": prints more detailed information when set to "DETAILED"; the default value is "CONCISE".

3.1.1 Pure RAM Configuration File

The pure RAM configuration file models an untagged structure. You can control the following parameters:

- size (in bytes): total size of your structure
- block size (in bytes): this is the size of an entry. Caution! Unfortunately, the tool requires the block size to be in bytes. If your entry size is not divisible by eight, then round it up to the next byte. Make sure you use this *rounded* block size when you specify the total size of your structure (previous parameter).
  As a reminder: *cache size = #sets * associativity * block size*. The number of sets should match the number of rows in your predictor.
- bus width (in bits): this should be at least as big as the block size.
- associativity: note that a fully-associative, untagged structure is not meaningful here.
- Number of read, write or read/write ports: A single read/write port should be sufficient for your purpose, but feel free to modify this.

The generated statistics of interest are listed below:

```
Access time (ns): 0.279886
Cycle time (ns):  0.277706
Total dynamic read energy per access (nJ): 0.0022076
Total leakage power of a bank (mW): 2.87418
Cache height x width (mm): 0.133416 x 0.0953244
```

You can compute the total area of the structure from the provided cache height and width. Access time (in ns) reflects the access time for your structure, whereas cycle time is the minimum time between the start times of two consecutive accesses. Usually cycle time is larger than access time, by a small percentage, but it can also be slightly smaller in pipelined circuits. For this assignment, you can focus on access time values. Finally, you can find the dynamic read energy per access and the leakage power of your structure.

Absolute numbers are useful, but it can be equally interesting to compare how these values change for different organizations.

3.1.2 Cache Configuration File

The cache configuration file models a tagged structure, similar to a cache. We will use the modeled tag array to estimate area, power, and energy of the used predictor structures.

The benefit of this approach is that we can control the width of the tag array. Therefore, if we have an entry with 4 history bits, we will not need to round this up to a byte. However, tag arrays involve (unnecessary for our purpose) tag comparators, which the pure RAM implementation lacked. Yet another design trade-off to consider.

To use the cache configuration file do the following:

1. Specify the size (in bytes) of the data array. With a block size of one, this will control the number of entries in your structure. We assume an associativity of one. In general, you need to specify the cache size, block size and associativity so that the data array (and thus the tag array that we care about) has the same number of sets (rows) as your predictor.

2. Specify the tag size. CACTI gives us the option to specify this in bits, so it can be arbitrarily small. For now we set it to eight bits, to be comparable with the pure RAM example.

Note that the cache access type is set to sequential, i.e., we first access the tag and then the data array. We have also set the "Print Level" option to Detailed, as we are only interested in the tag array component. The following statistics are of interest:

```
Tag side (with Output driver) (ns): 0.279886

Tag array:  Total dynamic read energy/access (nJ): 0.00203752
       Total leakage read/write power of a bank (mW): 2.87883

Tag array: Area (mm2): 0.0125494
       Height (mm): 0.131624
       Width (mm): 0.095343
```

As you can see, the generated numbers are comparable. Use the configuration file that is the most applicable to your design. If your design contains multiple tables, use multiple configuration files and decide how to combine all their outputs, e.g. are the tables accessed sequentially or in parallel? Remember to list all configuration parameters you modified in your report.

4 Methodology

The ChampSim [2] simulator provides a simple framework for implementing and comparing different branch predictors (and prefetchers in Lab 4). Our simulator is forked (and further simplified) from ChampSim and our benchmarks are a subset of those used in the 2026 4th Data Prefetching Championship (DPC4). Modify the simulator to measure the MPKI for all three dynamic branch prediction schemes. Then use the collected statistics to prepare a brief report as described in Section 6. In addition, use CACTI to provide area, delay, and power comparisons for the two-level and the open-ended branch predictors.

4.1 Compiling the simulator

Obtain and set up the ChampSim source code in your ug account with these Unix commands:

```
cd ~/ece552
cp /cad2/ece552f/ChampSim.tgz .
tar -zxf ChampSim.tgz
```

That sequence of commands extracts the simulator files into your working directory (`~/ece552/ChampSim`). These instructions assume that you already created the `ece552` directory for the Pre-Assignment. Do not leave the Unix permissions to your code open.

You can then build the simulator for a baseline system with *no pred*ictor using the provided `Makefile`:

```
cd ~/ece552/ChampSim
./config.sh ece552/nopred.json
make
```

4.2 Running the simulator

After building, you can run each ChampSim benchmark as follows:

```
cd ~/ece552/ChampSim

bin/nopred --warmup-instructions 5000000 --simulation-instructions 20000000 \
   /cad2/ece552f/Champsim_benchmarks/<trace-file>
```

Replace `<trace-file>` with a file in `/cad2/ece552f/Champsim_benchmarks/` such as `llama2.champsim.gz`.

On each run, the simulator executes only the branch predictor it was configured with before compilation. Each configuration file produces a different executable with a different predictor configured: `twobitsat`, `twolvl` and `openbranch`. Included in the simulator output is the line:

```
CPU 0 Branch Prediction Accuracy: <Br> Branches <Mis> Mispredicts <Acc>% \
MPKI: <MPKI> Average ROB Occupancy at Mispredict: <occ>
```

The `MPKI` (Mispredicts per thousand instructions) is the statistic we are interested in.

4.3 Modifying the simulator

You will only make changes to `branch/<predictor>/ece552_<predictor>.cc`. Do not modify or add any other files; they will not be taken into account.

In `ece552_<predictor>.cc`, you are provided with the following skeleton functions for you to implement each of the three branch prediction schemes (`<predictor>` refers to `2bitsat`, `2level` or `openend`):

- `initialize_branch_predictor`: Called only once, before the benchmark begins. Use this function to set up your data structures.
- `predict`: Called on each conditional branch instruction, **before** the branch direction and target address are resolved. Use this function to generate your prediction: return `true` if you predict that the branch will be taken, or `false` otherwise. Based on your return values,

the simulator counts the number of mispredicted branches and computes the MPKI for you. Note that by default, this function implements a static always-not-taken predictor, since it does nothing but return the value `false`. This function's only parameter is `ip`, which is the branch instruction pointer. Note that because the traces are x86 instructions, which are variable width, the lowest-order bits of the ip are *not* constant.

- `update`: Called on each conditional branch instruction, **after** the branch prediction and target address are resolved. Use this function to update your data structures with the branch outcome. This function takes in two parameters:

  1. `ip`: the branch instruction pointer.
  2. `resolveDir`: the resolved branch direction is `true` iff the branch was taken or `false` otherwise.

Do not modify the function names and parameters; they need to match the function declarations in `<predictor>.h`.

4.4 Microbenchmarking

This section describes how to run a microbenchmark program on the ChampSim simulator. Write your microbenchmark in a file `mb.c`. Please refer to the Assignment 1 handout for good practices on writing microbenchmarks. Compile your microbenchmark with the system `gcc` compiler:

```
gcc mb.c -o mb
```

To run your microbenchmark on the simulator, you first need to generate a trace file. To do this, ChampSim includes a Pintool [5] for generating a trace from a program's execution. Run the tool with your microbenchmark via the following commands:

```
cd ~/ece552/ChampSim
cd tracer/pin
make

/cad2/ece552f/pin3.22 -t obj-intel64/champsim_tracer.so \
   -o <trace_filename> \
   [-s # of instructions to skip] [-t # of instructions to trace] \
   -- <path/to/mb> [args of mb]
```

The tracer tool generates the trace file ¡trace_filename¿ in your working directory. This file stores the dynamic stream of all instructions executed by your microbenchmark. For example, you could generate a short trace of running `ls /bin` with:

```
/cad2/ece552f/pin3.22 -t obj-intel64/champsim_tracer.so \
    -o ls.champsim -s 10000 -t 10000 \
    -- /bin/ls /bin
```

You can now run the ChampSim simulator on your generated trace with

```
cd ~/ece552/ChampSim
bin/<predictor> tracer/pin/<trace_filename>
```

or for the concrete examples above:

```
bin/nopred tracer/pin/ls.champsim
```

5 Prelab

The prelab is worth 0.5/5 of the overall lab mark. Please complete the following steps before coming to the lab:

- Read all necessary background on branch prediction (textbook, lecture slides).
- Answer the following questions:
  1. Why do we use predictors with 2-bit saturating counters (i.e., bimodal)? Contrast this with the 1-bit approach.
  2. You are given the following code snippet, with two conditional branches clearly marked:

```
int a;
for (int i = 0; i < 100000; i++) { // conditional branch B1
    if ((i % 4) == 0) { // conditional branch B2
       a = 10;
    }
    a = 15;
 }
```

     (a) Write down the taken/not-taken sequence for the second conditional branch B2. List any assumptions you make about how this if-statement translates in assembly.
     (b) Assume you are keeping track of prior branch history for conditional branch B2. What is the smallest number of history bits (h) you need to correctly predict all B2's branch outcomes?
     (c) Assume you have a PAg predictor (Figure 3.20) with an eight-entries BHT and h history bits per entry (h is the previously computed number). Why would it be a bad idea to index the BHT with the 3 most significant PC bits?
- Write most of your code, along with some microbenchmarks, before coming to the lab.
- Finally, be prepared to answer any high-level questions about the simulator, your code, and the lab material (e.g., compare different branch prediction schemes).

6 Lab Deliverables

At the end of this assignment you should submit the following files using the `submitece552f` command:

1. `ece552_<predictor>.cc`: the modified ChampSim simulator files.
2. `mb.c`: your microbenchmark code for the two-level predictor. Your microbenchmark must include comments and snippets of generated assembly code that explain how it verifies the collected statistics. Reread the good microbenchmark practices from Assignment 1. Failure to include appropriate explanation will result in a grade of 0 for the microbenchmark portion of the assignment.
3. **report.pdf**: a brief two-page pdf report (single-spaced with 12-point font size). Make sure your report can be viewed on the ug machines through `xpdf` or `acroread`.
   - Your report should include both you and your lab partners names.

- Briefly explain how your microbenchmark collected statistics to validate the correctness of your code for the two-level predictor. Refer to specific branches in your code and their branch history. Feel free to refer to comments within the mb.c file, as needed. Specify which compilation flags you used.
- Provide a table with the number of mispredicted branches and the MPKI for all three predictors and benchmarks. Be wary of the two-page limit, since this table may be large.
- Describe your open-ended branch predictor implementation. This section must include how you calculated the storage requirements (in bytes) for your open-ended predictor.
- Report area, access latency, and leakage power for the two-level and your open-ended branch predictors, using CACTI. Specify the configuration parameters you modified.
- Include a brief statement of work completed by each partner.

4. **open-ended-bpred.cfg**: the CACTI configuration file you used for your open-ended predictor. It should be based either on cache.cfg or pureRAM.cfg. If you use multiple predictor tables, you can submit multiple configuration files with one for each table. For naming, please add the suffix `-1`, `-2`, etc. (e.g. `open-ended-bpred-1.cfg`). Be sure to clarify in your report what each `.cfg` file models.

5. **2level-bpred.cfg**: the CACTI configuration file for your two-level predictor.

The submit command is the following:

```
submitece552f 2 branch/*/ece552_*.cc report.pdf mb.c open-ended-bpred.cfg 2level-bpred.cfg
```

You can view the files that you have submitted via the command

```
submitece552f -l 2
```

Only one lab member from each group should submit code. Submissions from both partners will result in added confusion during marking.

Do not leave the Unix permissions open to your code or micro-benchmarks. You are strongly encouraged to submit your code early to determine how your open-ended predictor ranks compared to your classmates.

7 Due Date and Marking Scheme

This assignment is due on **Friday October 23, 2026 at 6:00pm EDT**. It is worth **5%** of the total course mark. The pre-lab will constitute 0.5/5 of the overall lab mark.

In order to receive any points on the open-ended predictor, your predictor must have less than 3.35 geo-mean MPKI across the eight benchmarks provided. Marks will be awarded based on the performance of your predictor and how adequately you describe your predictor in the report. Additional bonus points will be awarded to the top $\sim$10 performing open-ended predictors. During the week prior to the due date, you can submit your code each day; each night the TA will run the open-ended predictor and post a ranking of the results on Quercus. You can continue to update and resubmit your code until the deadline. Please plan ahead and start early!

An automarker will be used to compile and run your implementation and verify the statistics generated. Therefore, your implementation is required to follow some strict rules. To begin with, use the simulator package specifically provided for this assignment as described in Section 4.

Things to watch for:

- run the benchmarks as specified in the handout
- initialize properly all the data structures
- the only files to modify are `ece552_<predictor>.cc`
- do not add additional files, they will not be taken into account; your simulator should compile with the provided `Makefile` by typing `./config.sh ece552/<predictor>.json; make`
- an automatic comparison of your submissions will be done to check for copied code; changing variable names and formatting will not defeat the checker. Using code found on the internet is also prohibited.

8 Questions

Please post clarification questions on Piazza. Do not post solution code or microbenchmark code. This constitutes cheating and will not be tolerated.

References

[1] Rajeev Balasubramonian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. ACM Transactions on Architecture and Code Optimization vol. 14, no 2. June 2017.

[2] Gober, N., Chacon, G., Wang, L., Gratz, P. V., Jimenez, D. A., Teran, E., Pugsley, S., and Kim, J. (2022). The Championship Simulator: Architectural Simulation for Education and Competition. https://doi.org/10.48550/arXiv.2210.14324

[3] Michel Dubois, Murali Annavaram, and Per Stenström. Parallel Computer Organization and Design. 1st ed. Cambridge: Cambridge University Press. 2012. https://uoft.me/dubois-annavaram-stenstrom

[4] Michel Dubois, Murali Annavaram, and Per Stenström. Parallel Computer Organization and Design. 2nd ed. Cambridge: Cambridge University Press. 2025. https://go.openathens.net/redirector/utoronto.ca?url=https://doi.org/10.1017/9781009447621

[5] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In Proc. ACM SIGPLAN conference on Programming language design and implementation (PLDI). 2005.

[6] Naveen Muralimanohar, Rajeev Balasubramonian, and Norm Jouppi. Optimizing NUCA Organizations and Wiring Alternatives for Large Caches with CACTI 6.0. In Proc. IEEE/ACM International Symposium on Microarchitecture (MICRO-40). 2007.

[7] Teja Singh, Spence Oliver, Sundar Rangarajan, Shane Southard, Carson Henrion, et al. Zen 5: The AMD High-Performance 4nm x86-64 Microprocessor Core. In Proc. IEEE International Solid-State Circuits Conference (ISSCC). 2025.

[8] Steven J. E. Wilton and Norman P. Jouppi. CACTI: an enhanced cache access and cycle time model. IEEE Journal of Solid-State Circuits vol. 31, no. 5. May 1996.